



รายงาน

PS04: Memory class and API in C programming

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง	1
1 ข้อ 1: Static Storage Class	1
1.1 พื้นฐานที่ต้องเข้าใจก่อน: storage class คืออะไร	1
1.2 Step 1: โปรแกรมที่มี static คอมไพล์แบบปกติ	2
1.2.1 ผลการทดลอง	2
1.2.2 อธิบายผล	3
1.3 Step 2-3: ลบคำว่า static ออก	3
1.3.1 ผลการทดลอง	3
1.3.2 อธิบายผลและเปรียบเทียบกับ Step 1	3
1.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie	4
1.4.1 ผลการทดลอง	4
1.4.2 ต่างจากเดิมหรือไม่ เพราะอะไร	5
1.5 หลักฐานยืนยันคำอธิบาย	5
1.6 อภิปรายผล	7
2 ข้อ 2: Extern Storage Class	8
2.1 พื้นฐานที่ต้องเข้าใจก่อน: การประกาศ กับ การนิยาม	8
2.2 Step 1: โปรแกรมที่มี extern ทั้งสองฟังก์ชัน	8
2.2.1 ผลการทดลอง	9
2.2.2 อธิบายผล	9
2.3 Step 2-3: ลบคำว่า extern ในฟังก์ชัน main()	9
2.3.1 ผลการทดลอง	9
2.3.2 อธิบายผลและเปรียบเทียบกับ Step 1	10
2.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie	11
2.4.1 ต่างจากเดิมหรือไม่ เพราะอะไร	11
2.5 หลักฐานยืนยันคำอธิบาย	12
2.6 อภิปรายผล	14

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11), kernel 6.6.87.2-microsoft-standard-WSL2
สถาปัตยกรรม	x86-64 (64 บิต)
คอมไพเลอร์	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1) ตั้งค่ามาพร้อม <code>--enable-default-pie</code>
เครื่องมือตรวจสอบ	binutils (nm, readelf), file, util-linux (setarch)
ค่า ASLR	/proc/sys/kernel/randomize_va_space = 2 (สุ่มเต็มรูปแบบ)
Shell	GNU bash 5.2.21(1)-release
วันที่เก็บผล	6 กันยายน 2569

ผลการทดลองดิบ ทุกบรรทัดที่ยกมาในรายงานนี้ตัดมาจากไฟล์ `result/q1_verified.txt` และ `result/q2_verified.txt` ซึ่งสร้างซ้ำได้ด้วย `make result` (สคริปต์ `run_q1.sh` และ `run_q2.sh` ในโฟลเดอร์ `src`) โจทย์กำหนดว่า ทุกขั้นตอนต้องรันบนเครื่องเดียวกัน สคริปต์ของแต่ละข้อจึงคอมไพล์และรัน ทั้งสี่กรณีของข้อนั้น (แก้หรือไม่แก้โค้ด คุณกับ มีหรือไม่มี `-no-pie`) รวดเดียวจบในการเรียกครั้งเดียว ในรายงานแสดงเฉพาะบรรทัดที่จำเป็นต่อการอภิปราย ส่วนที่ตัดออกจะมีเครื่องหมาย ... หรือหมายเหตุ (lines omitted) กำกับไว้ และเพื่อให้ผลการรันสามครั้งอยู่ในหน้าเดียวกันได้ ได้ย้ายบรรทัดที่พิมพ์ *ที่อยู่* ขึ้นมาต่อท้ายบรรทัดที่พิมพ์ *ค่า* ของรอบเดียวกัน โดยไม่แก้ตัวเลขใด ๆ

หมายเหตุสองเรื่องเกี่ยวกับวิธีทดลอง (1) รายงานนี้คอมไพล์ด้วย `-Wall -Wextra` ตลอด จึงมี warning `return type of main is not int [-Wmain]` ติดมาทุกครั้ง เพราะโค้ดที่โจทย์ให้มา ประกาศ `void main()` ซึ่งไม่ตรงมาตรฐาน C (มาตรฐานกำหนดให้ `main` คืนค่า `int`) รายงานคงโค้ดไว้ตามโจทย์ทุกตัวอักษร เพราะเป็นเพียง warning ไม่ใช่ error และไม่กระทบผลการทดลองเรื่องที่อยู่ของตัวแปรเลย (2) การอธิบายว่า “ที่อยู่เปลี่ยนเพราะ ASLR” จะเชื่อถือได้ก็ต่อเมื่อ ปิด ASLR แล้วที่อยู่ต้องหยุดเปลี่ยน สคริปต์จึงรันโปรแกรมชุดเดิมซ้ำด้วย `setarch -R` (ปิดการสุ่ม address space เฉพาะ process ลูก) ไว้เป็นกรณีควบคุมในหัวข้อ 4 ของไฟล์ผล

1 ข้อ 1: Static Storage Class

1.1 พื้นฐานที่ต้องเข้าใจก่อน: storage class คืออะไร

คำว่า *storage class* ในภาษา C ไม่ได้บอกว่าตัวแปร “เป็นชนิดอะไร” แต่บอกว่าตัวแปรนั้น ถูกเก็บไว้ที่ไหนในหน่วยความจำ และมีชีวิตอยู่ยาวนานแค่ไหน คุณสมบัติที่ต้องแยกให้ออกมีสามเรื่อง

- ขอบเขตการมองเห็น (scope) คือบริเวณในโค้ดที่เรียกชื่อตัวแปรนี้ได้
- อายุ (lifetime) คือช่วงเวลาขณะที่ในหน่วยความจำของตัวแปรยังถูกจองไว้
- การเชื่อมโยง (linkage) คือไฟล์อื่นเห็นชื่อนี้ด้วยหรือไม่

หัวใจของข้อนี้คือ **scope กับ lifetime เป็นคนละเรื่องกัน** ตัวแปรหนึ่งตัว มีขอบเขตแคบมาก (เห็นได้เฉพาะในบล็อกเดียว) แต่มีอายุยืนตลอดโปรแกรมได้ และนั่นคือสิ่งที่คำว่า *static* ทำ

Table 1: เปรียบเทียบ storage class ของตัวแปรในภาษา C

ประกาศแบบ	ขอบเขต	อายุ	เก็บที่	ค่าเริ่มต้นถ้าไม่ได้
int y; ในบล็อก (คือ auto)	เฉพาะบล็อกนั้น	เกิดตอนเข้าบล็อก ตายตอนออกจากบล็อก	stack	ค่าขยะที่ค้างอยู่
static int y; ในบล็อก	เฉพาะบล็อกนั้น	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ
static int y; นอกฟังก์ชัน	ทั้งไฟล์	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ
int y; นอกฟังก์ชัน	ทั้งโปรแกรม	ตลอดโปรแกรม	.data หรือ .bss	0 เสมอ

อีกจุดที่ควรรู้คือคำว่า auto ในบรรทัด auto int x = 3; ของโจทย์ **ไม่ได้ทำอะไรเลย** เพราะตัวแปรที่ประกาศในบล็อกเป็น auto อยู่แล้วโดยปริยาย เขียนหรือไม่เขียนผลลัพธ์เหมือนกันทุกประการ (ในมาตรฐาน C23 คำนี้ถูกนำกลับมาใช้ในความหมายใหม่ คือให้คอมไพเลอร์เดาชนิดตัวแปรเอง แต่ gcc 13 ที่ใช้ในการทดลองนี้ตั้งต้นเป็น -std=gnu17 จึงยังเป็นความหมายเดิม)

1.2 Step 1: โปรแกรมที่มี static คอมไพล์แบบปกติ

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void main()
5 {
6     auto int x = 3;
7
8     while (x > 0)
9     {
10         static int y = 5;
11         y++;
12
13         printf("The value of y is %d\n", y);
14         printf("The address of y is %p\n\n", &y);
15
16         x--;
17     }
18 }

```

Code 1: src/q1_static.c คัดลอกจากโจทย์ทุกตัวอักษร

1.2.1 ผลการทดลอง

```

$ gcc -Wall -Wextra -o q1_static q1_static.c
q1_static.c:10:6: warning: return type of 'main' is not 'int' [-Wmain]
   10 | void main()
      |      ^~~~
(exit status = 0)

$ ./q1_static                                     # run 1
The value of y is 6      The address of y is 0x5c5469622010
The value of y is 7      The address of y is 0x5c5469622010
The value of y is 8      The address of y is 0x5c5469622010

$ ./q1_static                                     # run 2
The value of y is 6      The address of y is 0x60a5f69ee010
The value of y is 7      The address of y is 0x60a5f69ee010
The value of y is 8      The address of y is 0x60a5f69ee010

$ ./q1_static                                     # run 3
The value of y is 6      The address of y is 0x5faa56b46010
The value of y is 7      The address of y is 0x5faa56b46010
The value of y is 8      The address of y is 0x5faa56b46010

```

Code 2: คอมไพล์แบบปกติแล้วรัน 3 ครั้ง

1.2.2 อธิบายผล

ผลที่ได้มีสองส่วนที่ต้องอธิบายแยกกัน คือ *ค่า* กับ *ที่อยู่*

ค่าเป็น 6, 7, 8 ไม่ใช่ 6, 6, 6 เพราะบรรทัด `static int y = 5;` ไม่ใช่คำสั่งที่ทำงานตอนวนลูป แต่เป็นการบอกคอมไพเลอร์ให้จองเนื้อที่ไว้ในส่วน `.data` ของไฟล์โปรแกรม พร้อมเขียนเลข 5 ลงไปในไฟล์ตั้งแต่ ตอนคอมไพล์ เมื่อโปรแกรมถูกโหลดขึ้นหน่วยความจำ ค่า 5 ก็ติดมากับส่วนนั้นเลย การ “กำหนดค่าเริ่มต้น” จึงเกิดขึ้น **ครั้งเดียวก่อน** โปรแกรมเริ่มทำงาน ไม่ได้เกิดซ้ำๆ รอบที่ผ่านบรรทัดนั้น พอวนลูปรอบที่สอง โปรแกรมข้ามบรรทัดประกาศ ไปทำ `y++` ต่อจากค่าเดิมทันที ค่าจึงสะสมเป็น $5 \rightarrow 6 \rightarrow 7 \rightarrow 8$

ที่อยู่เท่ากันทั้ง 3 รอบภายในการรันเดียว เป็นผลโดยตรงจากข้อแรก คือมีตัวแปร `y` อยู่ *ตัวเดียว* ตลอดโปรแกรม ไม่ได้ถูกสร้างใหม่ทุกรอบ เมื่อมีตัวเดียวกีย่อมมีที่อยู่เดียว

แต่ที่อยู่เปลี่ยนทุกครั้งที่รันใหม่ ทั้งที่เป็นไฟล์โปรแกรมเดียวกัน สาเหตุคือ Ubuntu ตั้งค่า `gcc` ให้สร้าง **PIE (Position-Independent Executable)** เป็นค่าตั้งต้น (ยืนยันได้จาก `gcc -v` ที่มี `--enable-default-pie`) ไฟล์แบบนี้ไม่ได้ผูกที่อยู่จริงไว้ในตัวไฟล์ แต่เก็บเป็น *ระยะห่างจากจุดเริ่มของโปรแกรม* เอาไว้ เคอร์เนลจึงเลือก *ฐานที่* โหลด ใหม่แบบสุ่มได้ทุกครั้งที่รัน ซึ่งเป็นกลไก **ASLR (Address Space Layout Randomization)** ที่เปิดอยู่ในเครื่องนี้ (`randomize_va_space` มีค่า 2) ที่อยู่ที่พิมพ์ออกมาจึงเป็น “ฐานที่สุ่มได้ บวกกับ ระยะห่างคงที่” หลักฐานอยู่ในหัวข้อ 1.5

1.3 Step 2–3: ลบคำว่า `static` ออก

แก้ไขบรรทัดเดียวคือลบคำว่า `static` ออก (ไฟล์ `src/q1_auto.c`) ส่วนอื่นเหมือนเดิมทุกตัวอักษร

```
1 while (x > 0)
2 {
3     int y = 5;          /* the word "static" is gone */
4     y++;
5     ...
```

Code 3: `src/q1_auto.c` เฉพาะส่วนที่เปลี่ยน

1.3.1 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_auto q1_auto.c
$ ./q1_auto                                # run 1
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0
The value of y is 6      The address of y is 0x7fff16657ab0

$ ./q1_auto                                # run 2
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730
The value of y is 6      The address of y is 0x7ffde6d8b730

$ ./q1_auto                                # run 3
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0
The value of y is 6      The address of y is 0x7ffd35c62ec0
```

Code 4: ลบ `static` ออกแล้วรัน 3 ครั้ง

1.3.2 อธิบายผลและเปรียบเทียบกับ Step 1

ค่ากลายเป็น 6, 6, 6 เพราะตอนนี้ `y` เป็นตัวแปรอัตโนมัติ อายุของมันผูกกับ *บล็อก* ไม่ใช่โปรแกรม ทุกครั้งที่วนเข้าบล็อกใหม่ ตัวแปรจะถูกสร้างใหม่และคำสั่ง `= 5` ทำงานจริงทุกรอบ (ต่างจาก `static` ที่เป็นการเขียนค่าลงไฟล์ตั้งแต่ตอนคอมไพล์) พอบวกหนึ่งจึงได้ 6 เท่ากันทุกรอบ ค่าที่สะสมไว้รอบก่อนหายไปหมดตอนออกจากบล็อก

ที่อยู่ยังเท่ากันทั้ง 3 รอบ แต่เป็นคนละเหตุผลกับ Step 1 จุดนี้เป็นจุดที่เข้าใจผิดกันบ่อยที่สุด ที่อยู่เท่ากันไม่ได้แปลว่าเป็นตัวแปรตัวเดิม ความจริงคือคอมไพเลอร์จองสล็อตให้ y ไว้ใน *stack frame* ของ *main* ตั้งแต่ตอนเข้าฟังก์ชัน (เช่น “ถอยจากยอด frame ไป 20 ไบต์”) และ frame ของ *main* ไม่ได้ถูกทำลายระหว่างวนลูป การวนซ้ำออกบล็อกรวมกลับมาใช้ **สล็อตเดิมซ้ำ** ทุกรอบ ที่อยู่จึงบังเอิญเท่ากัน แต่ **เนื้อหา** ในสล็อตนั้น ถูกเขียนทับด้วยเลข 5 ใหม่ทุกรอบ ซึ่งเห็นได้จากค่าที่ไม่สะสม ถ้าย้ายบล็อกนั้นไปอยู่ในฟังก์ชันที่เรียกตัวเองซ้อนกัน (recursive) ที่อยู่ก็จะต่างกันไปทุกครั้งทันที เพราะแต่ละครั้งได้ frame คนละอันบน stack

ที่อยู่ยังเปลี่ยนทุกครั้งทีรันใหม่ และอยู่คนละย่านกับ Step 1 สังเกตว่าเลขของ Step 1 ขึ้นต้นด้วย $0x5...$ แต่ของ Step 2 ขึ้นต้นด้วย $0x7ff...$ ซึ่งไม่ใช่ความบังเอิญ เลข $0x7ff...$ คือย่านบนสุดของ address space ที่ระบบปฏิบัติการวาง *stack* ไว้ ส่วน $0x5...$ คือย่านที่ เคอร์เนลใช้โหลดตัวโปรแกรมแบบ PIE หลักฐานว่าเลขนี้เป็น *stack* จริง อยู่ในหัวข้อ 1.5 ส่วนสาเหตุที่เปลี่ยนทุกครั้งก็คือ ASLR เช่นกัน เพียงแต่เป็น **การสุ่มตำแหน่ง stack** ซึ่งเป็นคนละกลไกกับการสุ่มฐานของ PIE ประเด็นนี้จะสำคัญมากใน Step 4

Table 2: สรุปความต่างระหว่างมีและไม่มี static

ประเด็น	static int y = 5;	int y = 5;
ค่าที่พิมพ์ออกมา	6, 7, 8	6, 6, 6
กำหนดค่าเริ่มต้นเมื่อไร	ครั้งเดียว ตอน คอมไพล์ และ โหลดโปรแกรม	ทุกครั้งที่วนเข้าบล็อก
จำนวนตัวแปรจริง ๆ	1 ตัวตลอดโปรแกรม	สร้างใหม่ทุกรอบ แต่ใช้ที่เดิมซ้ำ
อยู่ ส่วนไหน ของ หน่วยความจำ	.data (ย่าน $0x5...$ เมื่อเป็น PIE)	stack (ย่าน $0x7ff...$)
อายุ	ตลอดโปรแกรม	เฉพาะช่วงที่อยู่ในบล็อก
ขอบเขตการมองเห็น	เฉพาะในบล็อก (เหมือนกันทั้งคู่)	เฉพาะในบล็อก (เหมือนกันทั้งคู่)
มีชื่อในตาราง symbol ใหม่	มี ชื่อ $y.0$	ไม่มี เป็นแค่ระยะทางใน frame

1.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie

ทำซ้ำ Step 1–3 ทั้งหมด โดยเปลี่ยนเฉพาะคำสั่งคอมไพล์เป็น `gcc -no-pie -o <output> <source>` ใช้ซอร์สโค้ดไฟล์เดิมทั้งสองไฟล์ ไม่แก้อะไรเลยแม้แต่ตัวอักษรเดียว

1.4.1 ผลการทดลอง

```
$ gcc -no-pie -Wall -Wextra -o q1_static_nopie q1_static.c
$ ./q1_static_nopie # run 1
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018

$ ./q1_static_nopie # run 2
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018

$ ./q1_static_nopie # run 3
The value of y is 6      The address of y is 0x404018
The value of y is 7      The address of y is 0x404018
The value of y is 8      The address of y is 0x404018
```

Code 5: -no-pie กับโปรแกรมที่มี static

```
$ gcc -no-pie -Wall -Wextra -o q1_auto_nopie q1_auto.c
$ ./q1_auto_nopie # run 1
The value of y is 6      The address of y is 0x7ffe931c00e0
```

```

The value of y is 6      The address of y is 0x7ffe931c00e0
The value of y is 6      The address of y is 0x7ffe931c00e0

$ ./q1_auto_nopie          # run 2
The value of y is 6      The address of y is 0x7fff0bc434d0
The value of y is 6      The address of y is 0x7fff0bc434d0
The value of y is 6      The address of y is 0x7fff0bc434d0

$ ./q1_auto_nopie          # run 3
The value of y is 6      The address of y is 0x7ffdbc93c020
The value of y is 6      The address of y is 0x7ffdbc93c020
The value of y is 6      The address of y is 0x7ffdbc93c020

```

Code 6: -no-pie กับโปรแกรมที่ไม่มี static

1.4.2 ต่างจากเดิมหรือไม่ เพราะอะไร

คำตอบคือ **ต่างเฉพาะครั้งเดียว** และครั้งที่ต่างกับครั้งที่ไม่ว่าหนึ่งเอง ที่ทำให้เห็นว่า -no-pie ทำงานกับหน่วยความจำส่วนไหนกันแน่

Table 3: สรุปผลทั้งสี่กรณี: ที่อยู่ของ y คงที่ข้ามการรันหรือไม่

กรณี	คอมไพล์ปกติ (PIE)	คอมไพล์ -no-pie	สรุป
มี static	0x5c54..., 0x60a5..., 0x5faa... เปลี่ยนทุกครั้ง	0x404018 เท่ากันทั้งสามครั้ง	ต่างกัน -no-pie ตรงที่อยู่ได้
ไม่มี static	0x7fff..., 0x7ffd..., 0x7ffd... เปลี่ยนทุกครั้ง	0x7ffe..., 0x7fff..., 0x7ffd... ยังเปลี่ยนทุกครั้ง	ไม่ต่าง -no-pie ไม่ช่วยอะไร
ค่าที่พิมพ์	6,7,8 และ 6,6,6	6,7,8 และ 6,6,6	ไม่ต่าง เหมือนเดิมทุกกรณี

(1) ค่าของ y ไม่เปลี่ยนแปลงไม่ว่าจะคอมไพล์แบบไหน เพราะ -no-pie เป็นเรื่องของ การจัดวางในหน่วยความจำ ล้วน ๆ ไม่ได้เปลี่ยน ความหมายของภาษา C แม้แต่ชนิดเดียว static ก็ยังคงเป็น static ตัวแปรอัตโนมัติก็ยังคงเกิดใหม่ทุกครั้งเหมือนเดิม

(2) กรณี static ต่างชัดเจน ที่อยู่กลายเป็น 0x404018 เท่ากันเป๊ะทั้งสามครั้ง เพราะ -no-pie สั่งให้ linker สร้างไฟล์ชนิด EXEC ที่ ผู้ที่อยู่จริงไว้ตายตัวตั้งแต่ตอน link เคอร์เนลไม่มีทางเลือก ต้องโหลดโปรแกรมลงที่อยู่นั้นเป๊ะ ๆ เสมอ ตัวแปร y ที่อยู่ในส่วน .data จึงมีที่อยู่เดิมทุกครั้ง ตรวจสอบได้ด้วย nm ว่าเลข 0x404018 ที่โปรแกรมพิมพ์ออกมา คือเลขเดียวกับที่อยู่ในตาราง symbol ของไฟล์ ไม่ได้ถูกกำหนดขึ้นตอนรันเลย

(3) กรณีไม่มี static ไม่ต่างเลย ที่อยู่ยังเป็น 0x7fff... และยังสุมทุกครั้งเหมือนเดิม เพราะ y ตัวนี้อยู่บน stack ซึ่ง ไม่ได้อยู่ในไฟล์โปรแกรม เลย stack เป็นสิ่งที่เคอร์เนลสร้างให้ ตอนสร้าง process และตำแหน่งของมันถูกสุมโดยกลไก ASLR ของเคอร์เนล ซึ่งเป็นคนละกลไกกับการสุมฐานของ PIE โดยสิ้นเชิง -no-pie จึงปิดได้ เฉพาะการสุมฐานของตัวโปรแกรม แต่ปิดการสุม stack ไม่ได้ ถ้าอยากปิดการสุม stack ด้วยต้องสั่งที่ระบบปฏิบัติการ เช่น setarch -R หรือแก้ค่า /proc/sys/kernel/randomize_va_space ซึ่งได้ทดลองไว้แล้วในหัวข้อถัดไป

ข้อสรุปของ Step 4 -no-pie ไม่ได้ “ปิด ASLR” อย่างที่มักเข้าใจกัน มันแค่ทำให้ ตัวโปรแกรมและข้อมูลของโปรแกรม (.text, .data, .bss) มีที่อยู่ตายตัว ส่วน stack, heap และไลบรารีที่โหลดตอนรัน ยังถูกสุมอยู่เหมือนเดิม

1.5 หลักฐานยืนยันคำอธิบาย

คำอธิบายทั้งหมดข้างต้นจะเป็นแค่การอ้างลอย ๆ ถ้าไม่มีการตรวจสอบจากอีกทาง สคริปต์เก็บผลจึงมีหัวข้อ 4 ที่ได้ตรวจสอบเรื่อง คือ ตัวไฟล์เป็นชนิดอะไร ตัวแปรอยู่ตรงไหนในไฟล์ และถ้าปิด ASLR แล้วผลจะเปลี่ยนไปอย่างไร

หลักฐานที่ 1: ไฟล์ที่ได้เป็นคนละชนิดกันจริง

```
$ file q1_static q1_auto q1_static_nopie q1_auto_nopie
q1_static:      ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...
q1_auto:        ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...
q1_static_nopie: ELF 64-bit LSB executable,      x86-64, dynamically linked, ...
q1_auto_nopie:  ELF 64-bit LSB executable,      x86-64, dynamically linked, ...

$ readelf -h q1_static | grep -E 'Type|Entry'
Type:          DYN (Position-Independent Executable file)
Entry point address: 0x1060      <- an offset, not a real address

$ readelf -h q1_static_nopie | grep -E 'Type|Entry'
Type:          EXEC (Executable file)
Entry point address: 0x401050    <- a real, fixed address
```

Code 7: ชนิดของไฟล์ ELF ทั้งสี่ (ตัด BuildID ออก)

หลักฐานที่ 2: ตัวแปร static มีชื่อและที่อยู่อยู่ในไฟล์ ส่วนตัวแปรอัตโนมัติไม่มี

```
$ nm q1_static_nopie | grep -i 'y'          # 'd' = a local symbol in .data
0000000000404018 d y.0                      <- exactly what the program printed

$ nm q1_static | grep -i 'y'               # PIE: the same symbol, but as an offset
0000000000004010 d y.0

$ nm q1_auto_nopie | grep -i 'y'           # nothing at all
(exit status = 1 -> 1 means grep found no symbol named y)

$ readelf -S q1_static_nopie | grep -A1 ' .data '
[25] .data          PROGBITS          0000000000004008 00003008
      0000000000000014 0000000000000000 WA          0      0      8
```

Code 8: ตาราง symbol และส่วน .data

บรรทัดแรกคือหลักฐานที่ตรงที่สุดของทั้งข้อ เลข 0x404018 ที่โปรแกรมพิมพ์ ออกมาตอนรัน คือเลขเดียวกับที่ nm อ่านได้จาก ตัวไฟล์ โดยไม่ต้องรันเลย และมันตกอยู่ในช่วงของส่วน .data ที่เริ่มที่ 0x404008 พอดี ส่วนชื่อ y.0 ที่มีเลขต่อท้าย เป็นเพราะคอมไพเลอร์ต้องตั้งชื่อไม่ให้ชนกับ static ตัวอื่นที่อาจชื่อ y เหมือนกันในฟังก์ชันอื่น ในทางกลับกัน q1_auto_nopie ไม่มี symbol ชื่อ y เลย เพราะตัวแปรอัตโนมัติไม่ใช่ “ของ” ที่อยู่ในไฟล์ แต่เป็นเพียงระยะห่างในโค้ดที่นำไปคำนวณกับ stack pointer ตอนรัน

หลักฐานที่ 3: ปิด ASLR แล้วที่อยู่หยุดสุ่มจริง (กรณีควบคุม)

```
$ setarch -R ./q1_static | grep address    # run 1, 2, 3
The address of y is 0x555555558010
The address of y is 0x555555558010
The address of y is 0x555555558010

$ setarch -R ./q1_auto | grep address      # run 1, 2, 3
The address of y is 0x7ffffffffffe290
The address of y is 0x7ffffffffffe290
The address of y is 0x7ffffffffffe290
```

Code 9: รันไฟล์ PIE ชุดเดิมด้วย setarch -R ซึ่งปิดการสุ่ม address space

ไฟล์เดียวกันกับที่เพิ่งให้ผลสุ่มไปเมื่อครู่ พอปิด ASLR กลับให้เลขเดิมทุกครั้ง นี่คือหลักฐานว่าตัวการที่ทำให้ที่อยู่เปลี่ยนคือ ASLR ไม่ใช่ตัวภาษา C และไม่ใช่ static ที่สำคัญคือ q1_auto ที่คอมไพล์แบบ PIE ก็หยุดสุ่มด้วย ทั้งที่ -no-pie ทำให้มันหยุดสุ่มไม่ได้ ยืนยันข้อสรุปของ Step 4 ว่าที่อยู่ของ stack ถูกคุมด้วย ASLR ของเคอร์เนล ไม่ใช่ด้วยชนิดของไฟล์ ELF

หลักฐานที่ 4: เลข 0x7ff... เป็นที่อยู่บน stack จริง

```
$ setarch -R ./q1_auto | grep address | head -1
The address of y is 0x7ffffffffffe290      <- inside the range below

$ setarch -R bash -c 'grep "[stack\]" /proc/self/maps'
7fffffde000-7ffffffffff00 rw-p 00000000 00:00 0          [stack]

$ setarch -R ./q1_static | grep address | head -1
The address of y is 0x555555558010          <- NOT inside that range
```

Code 10: เทียบกับแผนที่หน่วยความจำจริง (ปิด ASLR ทั้งคู่เพื่อให้เทียบตัวเลขกันได้)

ที่อยู่ 0x7fffffff290 ที่โปรแกรมพิมพ์ออกมา ตกอยู่ในช่วง 0x7fffffffde000 ถึง 0x7fffffff000 ที่เคอร์เนลระบุว่า เป็น [stack] พอดี ในขณะที่ที่อยู่ของตัวแปร static อยู่คนละย่านโดยสิ้นเชิง

หลักฐานที่ 5: ที่อยู่ของ PIE เท่ากับ ฐานที่สุ่มได้ บวก ระยะห่างคงที่

```
$ ./q1_static | head -2          # run 1, 2, 3 -> base = printed address - 0x4010
printed &y = 0x59dfb0606010      load base = 0x59dfb0602000
printed &y = 0x6234ef9d2010      load base = 0x6234ef9ce000
printed &y = 0x64fd3cae1010      load base = 0x64fd3cadd000

$ ./q1_static_nopie | head -2   # run 1, 2, 3 -> base = printed address - 0x404018
printed &y = 0x404018           load base = 0x0
printed &y = 0x404018           load base = 0x0
printed &y = 0x404018           load base = 0x0
```

Code 11: แยกส่วนที่สุ่มออกจากส่วนที่คงที่ (ระยะห่างของ y คือ 0x4010 จาก nm)

ข้อสังเกตสองอย่าง หนึ่งคือฐานที่คำนวณได้ลงท้ายด้วย 000 ทุกครั้ง เพราะเคอร์เนลสุ่มได้ที่ละหน้า (page ละ 4096 ไบต์) ไม่ได้สุ่มที่ละไบต์ สองคือไฟล์แบบ -no-pie มีฐานเป็นศูนย์ นั่นคือที่อยู่ที่พิมพ์ออกมา เท่ากับที่อยู่ที่ระบุไว้ในไฟล์ เป๊ะ ๆ ซึ่งเป็นนิยามของคำว่า “ไม่ position-independent”

1.6 อภิปรายผล

- **static เปลี่ยน “อายุ” ไม่ได้เปลี่ยน “ขอบเขต”**: เป็นบทเรียนหลักของข้อนี้ ตัวแปร y ในทั้งสองเวอร์ชันมีขอบเขตเท่ากันเป๊ะ คือเรียกใช้ได้เฉพาะในบล็อกของ while เท่านั้น โค้ดนอกบล็อกยังคอมไพล์ไม่ผ่านเหมือนกัน สิ่ง static เปลี่ยน คือย้ายตัวแปรจาก stack ไปอยู่ .data ทำให้อายุยืนตลอดโปรแกรม ผลคือได้ตัวแปรที่ “จำค่าได้ระหว่างการเรียกแต่ละครั้ง แต่ไม่มีใครนอกบล็อก ไปยุ่งกับมันได้” ซึ่งเป็นเหตุผลที่นิยมใช้ทำตัวนับหรือ cache ในฟังก์ชัน
- **การกำหนดค่าเริ่มต้นของ static เกิดตอนคอมไพล์ ไม่ใช่ตอนรัน**: เป็นสาเหตุที่ค่าออกมาเป็น 6, 7, 8 และเป็นเหตุผลที่มาตรฐาน C บังคับว่า ค่าเริ่มต้นของตัวแปร static ต้องเป็นค่าคงที่ที่รู้ตั้งแต่ตอนคอมไพล์ (เขียน static int y = rand(); ไม่ได้ แต่ int y = rand(); ได้) ผลข้างเคียงอีกอย่างคือถ้าไม่ใส่ค่าเริ่มต้น ตัวแปร static จะได้ค่า 0 เสมอโดยไม่ต้องเสียเนื้อที่ในไฟล์ เพราะไปอยู่ในส่วน .bss ที่เคอร์เนลล้างเป็นศูนย์ให้ตอนโหลด ต่างจากตัวแปรอัตโนมัติ ที่ถ้าไม่ใส่ค่าเริ่มต้นจะได้ค่าขยะที่ค้างอยู่ใน stack
- **ที่อยู่ที่พิมพ์ออกมาเป็น virtual address ไม่ใช่ที่อยู่จริงบน RAM**: เห็นได้ชัดจากกรณี -no-pie ที่ทุก process ได้เลข 0x404018 เหมือนกันหมด ถ้ารันพร้อมกันสิบตัวก็ยังเป็นเลขนี้ทั้งสิบตัวและไม่ชนกัน เพราะแต่ละ process มี address space ของตัวเอง โดยมีหน่วย MMU กับตาราง page ของเคอร์เนลแปลงเลขนี้ไปเป็นที่อยู่จริงคนละที่กัน นี่คือการอุปสรรคของเรื่อง virtual memory ที่เรียนในทฤษฎี
- **เหตุผลที่ Linux รุ่นใหม่ทำ PIE เป็นค่าตั้งต้น คือความปลอดภัย**: การโจมตีแบบ buffer overflow ยุคใหม่ (เช่น return-to-libc หรือ ROP) ต้องรู้ที่อยู่ของโค้ดหรือข้อมูลที่จะกระโดดไปให้แม่นยำ การสุ่มฐานทุกครั้ง ที่รันทำให้ผู้โจมตีต้องเดา ซึ่งมักทำให้โปรแกรมพังไปเสียก่อน แลกมาด้วยราคาเล็กน้อยคือโค้ดต้องอ้างอิงแบบ relative จึงเป็นเหตุผลที่ -no-pie ยังมีอยู่ คือใช้ตอน debug ที่อยากให้อยู่คงที่เพื่อเทียบผลได้ หรือใช้กับโค้ดเก่า ที่เขียนแบบผูกที่อยู่ตายตัว ไม่ใช่ตัวเลือกที่ควรใช้กับโปรแกรมจริง
- **ข้อควรระวังของ static ในโปรแกรมจริง**: เมื่อฟังก์ชันมี static อยู่ข้างใน ฟังก์ชันนั้นจะ “จำอดีต” ได้ ผลคือเรียกด้วย อาร์กิวเมนต์เดิมสองครั้งอาจได้ผลไม่เหมือนกัน ทำให้ทดสอบยากขึ้นมาก และที่ร้ายกว่านั้นคือถ้ามีหลาย thread เรียกฟังก์ชันเดียวกันพร้อมกัน ทุก thread จะใช้ตัวแปรตัวเดียวกัน เกิด race condition ทันที (ฟังก์ชันเก่าในไลบรารีมาตรฐานอย่าง strtok หรือ ctime มีปัญหานี้ จึงมีรุ่นลงท้ายด้วย _r ที่ให้ผู้เรียกสงที่เก็บสถานะมาเองแทน)
- **void main() ไม่ถูกต้องตามมาตรฐาน**: มาตรฐาน C กำหนดให้ main คืนค่า int เพื่อส่ง exit status ให้ระบบ

ปฏิบัติการ รายงานนี้คงได้ทำตามโจทย์และแนบ warning มาแสดงตามจริง แต่ในการใช้งานจริง ควรเขียนเป็น `int main(void)` และ `return 0;` เสมอ โดยเฉพาะเมื่อโปรแกรมจะถูกเรียกจากสคริปต์ที่ตรวจ exit status

2 ข้อ 2: Extern Storage Class

2.1 พื้นฐานที่ต้องเข้าใจก่อน: การประกาศ กับ การนิยาม

ข้อ 1 ว่าด้วย อายุ ของตัวแปร ส่วนข้อนี้ว่าด้วย การเชื่อมโยง (linkage) คือเรื่องที่ว่าชื่อตัวแปรหนึ่งชื่อ ไฟล์อื่นเห็นด้วยหรือไม่ และหัวใจของมันคือ การแยกให้ออกระหว่างสองคำที่คนมักใช้ปนกัน

- **การนิยาม (definition)** คือการสั่งให้ จองเนื้อที่จริง ในหน่วยความจำ เช่น `int x = 20;` ที่เขียนนอกฟังก์ชัน ตัวแปรหนึ่งตัวมี definition ได้เพียงทีเดียวในทั้งโปรแกรม
- **การประกาศ (declaration)** คือการบอกคอมไพเลอร์เพียงว่า “มีตัวแปรชื่อนี้ ชนิดนี้ อยู่ที่ไหนสักแห่ง เดียว linker ไปหามาให้” เช่น `extern int x;` ซึ่ง ไม่จองเนื้อที่ใด ๆ ทั้งสิ้น และเขียนซ้ำกี่ก็ได้

Table 4: การนิยามกับการประกาศ ต่างกันตรงไหน

ประเด็น	<code>int x = 20;</code> (นอกฟังก์ชัน)	<code>extern int x;</code>
เรียกว่าอะไร	definition (นิยาม)	declaration (ประกาศ)
จองเนื้อที่ไหม	จอง 4 ไบต์ในส่วน .data	ไม่จองเลย
เขียนซ้ำได้ไหม	ได้ทีเดียวในทั้งโปรแกรม	กี่ก็ได้
ใครเป็นคนจัดการ	คอมไพเลอร์	linker เป็นคนหาให้ตอนเชื่อมไฟล์
ถ้าไม่มีเลยจะเกิดอะไร	undefined reference ตอน link	คอมไพล์ผ่าน แต่อาจหา definition ไม่เจอ

จุดที่ต้องสังเกตในโค้ดของโจทย์คือ `extern int x;` ที่เขียนอยู่ ข้างใน `main()` และ `display()` นั้น อยู่ในไฟล์เดียวกันกับ `int x = 20;` ที่เป็น definition อยู่แล้ว ทั้งสองบรรทัดจึงเป็นการเขียนซ้ำซ้อนที่ **ไม่มีผลอะไรเลย** เพราะตัวแปร global มองเห็นได้จากทุกฟังก์ชันในไฟล์อยู่แล้ว ตั้งแต่บรรทัดที่นิยามมันลงไป ข้อนี้จึงน่าสนใจตรงที่ผลของการ ลบ คำว่า `extern` ออก ไม่ได้เป็นอย่างที่หลายคนคิดว่า “main จะมองไม่เห็น x” แต่กลับกลายเป็นอย่างอื่นไปเลย ซึ่งจะเห็นใน Step 2

2.2 Step 1: โปรแกรมที่มี extern ทั้งสองฟังก์ชัน

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int x = 20;
5
6 void display()
7 {
8     extern int x;
9     printf(" Display value of x: %d\n", x);
10    printf(" Address of x (in display function): %p\n", &x);
11 }
12
13 void main()
14 {
15     extern int x;
16     printf(" Print value of x: %d\n", x);
17     printf(" Address of x (in main function): %p\n", &x);
18
19     display();
20 }
```

Code 12: src/q2_extern.c คัดลอกจากโจทย์ทุกตัวอักษร

2.2.1 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q2_extern q2_extern.c
q2_extern.c:19:6: warning: return type of 'main' is not 'int' [-Wmain]
(exit status = 0)

$ ./q2_extern                                     # run 1
Print value of x: 20                               Address of x (in main function): 0x574a1053e010
Display value of x: 20                             Address of x (in display function): 0x574a1053e010

$ ./q2_extern                                     # run 2
Print value of x: 20                               Address of x (in main function): 0x650dd456f010
Display value of x: 20                             Address of x (in display function): 0x650dd456f010

$ ./q2_extern                                     # run 3
Print value of x: 20                               Address of x (in main function): 0x59201f88d010
Display value of x: 20                             Address of x (in display function): 0x59201f88d010
```

Code 13: คอมไพล์แบบปกติแล้วรัน 3 ครั้ง

2.2.2 อธิบายผล

ค่าเป็น 20 ทั้งสองที่ เพราะทั้ง main() และ display() กำลังพูดถึงตัวแปร **ตัวเดียวกัน** คือ x ที่นิยามไว้นอกฟังก์ชัน คำว่า extern ในทั้งสองฟังก์ชันเป็นเพียงการยืนยันว่า “ชื่อ x ที่ใช้ตรงนี้ หมายถึงตัวที่นิยามไว้ข้างนอก” ซึ่งเป็นสิ่งที่คอมไพเลอร์เข้าใจอยู่แล้วโดยไม่ต้องบอก

ที่อยู่ในสองฟังก์ชันตรงกันเป๊ะ นี่คือหลักฐานที่เข้มที่สุดว่ามีตัวแปร เพียงตัวเดียวจริง ๆ ถ้าเป็นคนละตัวย่อมเป็นไปได้ที่ที่อยู่จะตรงกันทุกบิต และตรงกันซ้ำแบบนี้ทั้งสามรอบ ข้อสังเกตนี้จะกลายเป็นเครื่องมือหลักในการอ่านผลของ Step 2 คือใช้ **ที่อยู่** เป็นตัวบอกว่ากำลังดูตัวแปรตัวเดียวกันอยู่หรือเปล่า

ที่อยู่เปลี่ยนทุกครั้งที่รันใหม่ ด้วยเหตุผลเดียวกับข้อ 1 ทุกประการ คือไฟล์เป็น PIE และเคอร์เนลสุ่มฐานที่โหลดใหม่ทุกครั้งตามกลไก ASLR สังเกตว่าเลขลงท้าย 010 เหมือนกันทั้งสามรอบ เพราะระยะห่างของ x จากฐานเป็นค่าคงที่ ส่วนที่เปลี่ยนคือฐานเท่านั้น

2.3 Step 2–3: ลบคำว่า extern ในฟังก์ชัน main()

แก้ไขตามโจทย์คือลบคำว่า extern ในบรรทัดที่ 15 ซึ่งอยู่ใน main() ออกเพียงคำเดียว (ไฟล์ src/q2_noextern.c) ส่วนใน display() คงไว้เหมือนเดิม

```
1 void main()
2 {
3     int x;                               /* the word "extern" is gone: this is now a NEW variable */
4     printf(" Print value of x: %d\n", x);
5     ...
```

Code 14: src/q2_noextern.c เฉพาะส่วนที่เปลี่ยน

2.3.1 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q2_noextern q2_noextern.c
q2_noextern.c:19:6: warning: return type of 'main' is not 'int' [-Wmain]
q2_noextern.c: In function 'main':
q2_noextern.c:22:5: warning: 'x' is used uninitialized [-Wuninitialized]
   22 |     printf(" Print value of x: %d\n", x);
      |     ^~~~~~
q2_noextern.c:21:9: note: 'x' declared here
   21 |     int x;
      |     ^
(exit status = 0)
```

Code 15: ลบ extern ใน main() แล้วคอมไพล์ คอมไพเลอร์เตือนเพิ่มมาหนึ่งข้อ

```

$ ./q2_noextern                               # run 1
Print value of x: 32767      Address of x (in main function): 0x7ffffbd75e6e4
Display value of x: 20      Address of x (in display function): 0x64d04c4a1010

$ ./q2_noextern                               # run 2
Print value of x: 32767      Address of x (in main function): 0x7ffff6f1a5714
Display value of x: 20      Address of x (in display function): 0x5595b064f010

$ ./q2_noextern                               # run 3
Print value of x: 32766      Address of x (in main function): 0x7ffe04a25a14
Display value of x: 20      Address of x (in display function): 0x579c5bb0f010

```

Code 16: ผลการรัน 3 ครั้ง

2.3.2 อธิบายผลและเปรียบเทียบกับ Step 1

สิ่งที่เกิดขึ้นไม่ใช่ “main มองไม่เห็น x” แต่คือ “main สร้าง x ตัวใหม่” นี่คือหัวใจของข้อนี้ พอตอบคำว่า extern ออก บรรทัดนั้นเปลี่ยนจาก *การประกาศ* ว่าจะใช้ตัวแปรข้างนอก กลายเป็น *การนิยาม* ตัวแปรอัตโนมัติตัวใหม่ชื่อ x ขึ้นมาใน main() ผลคือชื่อ x ในขอบเขตของ main ถูกจับจองโดยตัวใหม่นี้ และ **บดบัง (shadow)** ตัวแปร global เดิมจนมองไม่เห็น โปรแกรมยังคอมไพล์ผ่านและรันได้ตามปกติ เพราะในทางไวยากรณ์ไม่มีอะไรผิดเลย

ค่าที่พิมพ์ออกมาเป็นค่าขยะ ตัวแปรใหม่นี้เป็นตัวแปรอัตโนมัติที่ไม่ได้ใส่ค่าเริ่มต้น จึงได้ค่าที่ค้างอยู่ในสล็อตของ stack ตรงนั้น ซึ่งตามมาตรฐาน C ถือเป็น *undefined behavior* คอมไพเลอร์จับได้และเตือนไว้ ด้วย -Wuninitialized ตรงตำแหน่งบรรทัดที่ใช้งานพอดี้ ซึ่งเป็นเหตุผลที่ควร คอมไพล์ด้วย -Wall เสมอ

ค่าขยะที่ได้ไม่ได้สุ่มมั่ว แต่อธิบายได้ สังเกตว่าค่าที่ออกมาวนอยู่ แถว 32765–32767 ซึ่งใกล้ค่าสูงสุดของ short พอดี้ เมื่อเทียบกับที่อยู่ของ ตัวมันเองจะเห็นความสัมพันธ์ที่ตรงกันทุกรอบ

Table 5: ค่าขยะที่พิมพ์ออกมา เท่ากับ 16 บิตบนของที่อยู่บน stack พอดี้ทุกรอบ

การรัน	ค่าที่พิมพ์	ที่อยู่ของ x ใน main	16 บิตบนของที่อยู่
PIE รอบ 1	32767	0x7fff bd75e6e4	0x7fff = 32767
PIE รอบ 2	32767	0x7fff 6f1a5714	0x7fff = 32767
PIE รอบ 3	32766	0x7ffe 04a25a14	0x7ffe = 32766
-no-pie รอบ 1	32767	0x7fff 29c5a044	0x7fff = 32767
-no-pie รอบ 2	32766	0x7ffe ad28b514	0x7ffe = 32766
-no-pie รอบ 3	32765	0x7ffd 1d346e74	0x7ffd = 32765

ตรงกันทั้ง 6 รอบไม่ใช่ความบังเอิญ อธิบายได้ว่าสล็อตบน stack ที่ x ไปตกลงนั้น เคยถูกใช้เก็บ *ครึ่งบนของพอยน์เตอร์ 8 ไบต์ที่ชี้ไปบน stack* มาก่อน (โค้ดเริ่มต้นโปรแกรมและ dynamic loader ทำงานผ่านบริเวณนี้ก่อนถึง main) พอยน์เตอร์บน stack ของเครื่อง 64 บิตหน้าตาเป็น 0x00007fff_xxxxxxx เมื่ออ่านเฉพาะ 4 ไบต์บนของมันเป็น int จึงได้ 0x00007fff ซึ่งก็คือ 32767 และเนื่องจากตัวแปร x เองก็อยู่บน stack เหมือนกัน 16 บิตบนของทั้งสองค่าจึงตรงกันเสมอ ย้ำว่าเลขนี้ *ไม่ได้รับประกัน* เป็นเพียงสิ่งที่บังเอิญค้างอยู่ ถ้าเปลี่ยนเครื่อง เปลี่ยนคอมไพเลอร์ หรือแม้แต่เปลี่ยนความยาวของตัวแปรสภาพแวดล้อม ก็อาจได้เลขอื่น

display() ยังทำงานถูกต้องเหมือนเดิม เพราะไม่ได้ถูกแก้ มันยังเห็นตัวแปร global ตัวเดิม จึงยังพิมพ์ 20 และพิมพ์ที่อยู่ในย่าน 0x5... เหมือน Step 1 ทุกประการ จุดนี้คือการยืนยันว่าปัญหาเกิดเฉพาะ ในขอบเขตของ main ไม่ได้ลามไปทั้งโปรแกรม

ที่อยู่คนละย่านคือหลักฐานว่าเป็นคนละตัวแปร เทียบกับ Step 1 ที่ที่อยู่ในสองฟังก์ชันตรงกันเป๊ะ พอมาถึง Step 2 ที่อยู่ใน main เป็น 0x7ff... (stack) ส่วนใน display เป็น 0x5... (ส่วน .data ของโปรแกรม) ห่างกันคนละย่านของ address space ไปเลย

Table 6: สรุปความต่างระหว่างมีและไม่มี extern ใน main()

ประเด็น	extern int x; ใน main	int x; ใน main
บรรทัดนั้นคืออะไร	การประกาศ อ้างถึงตัวแปรข้างนอก	การนิยามตัวแปรใหม่ในบล็อก
มีตัวแปรกี่ตัว	1 ตัว ใช้ร่วมกันทั้งโปรแกรม	2 ตัว คือ global กับตัวใหม่ใน main
main พิมพ์ค่าอะไร	20	ค่าขยะ (32765-32767)
display พิมพ์ค่าอะไร	20	20 (ไม่กระทบ)
ที่อยู่ในสองฟังก์ชัน	ตรงกันเป๊ะ	คนละย่านโดยสิ้นเชิง
warning จาก gcc	ไม่มีเพิ่ม	'x' is used uninitialized
เก็บที่ไหน	.data ทั้งคู่	main อยู่ stack, display อยู่ .data

2.4 Step 4: คอมไพล์ใหม่ด้วย -no-pie

```
$ gcc -no-pie -Wall -Wextra -o q2_extern_nopie q2_extern.c
$ ./q2_extern_nopie # run 1, 2 and 3 are all identical
Print value of x: 20 Address of x (in main function): 0x404018
Display value of x: 20 Address of x (in display function): 0x404018
```

Code 17: -no-pie กับโปรแกรมที่มี extern ครบ

```
$ gcc -no-pie -Wall -Wextra -o q2_noextern_nopie q2_noextern.c
$ ./q2_noextern_nopie # run 1
Print value of x: 32767 Address of x (in main function): 0x7fff29c5a044
Display value of x: 20 Address of x (in display function): 0x404020

$ ./q2_noextern_nopie # run 2
Print value of x: 32766 Address of x (in main function): 0x7ffead28b514
Display value of x: 20 Address of x (in display function): 0x404020

$ ./q2_noextern_nopie # run 3
Print value of x: 32765 Address of x (in main function): 0x7ffd1d346e74
Display value of x: 20 Address of x (in display function): 0x404020
```

Code 18: -no-pie กับโปรแกรมที่ลบ extern ใน main()

2.4.1 ต่างจากเดิมหรือไม่ เพราะอะไร

คำตอบเหมือนข้อ 1 คือ ต่างเฉพาะส่วนที่อยู่ในไฟล์โปรแกรม แต่ข้อนี้ให้ภาพที่คมกว่า เพราะเห็นผลทั้งสองแบบ ในโปรแกรมเดียวกัน คือ q2_noextern_nopie ตัวเดียว มีตัวแปรสองตัวที่ตอบสนองต่อ -no-pie ไม่เหมือนกันเลย

Table 7: สรุปผลทั้งสี่กรณีของข้อ 2

ตัวแปรที่ดู	คอมไพล์ปกติ (PIE)	คอมไพล์ -no-pie	สรุป
x global (มี extern)	0x574a..., 0x650d..., 0x5920... เปลี่ยนทุกครั้ง	0x404018 ทั้งสามครั้ง	ต่างกัน ตรงที่อยู่ได้
x local ใน main (ลบ extern)	0x7fff..., 0x7fff..., 0x7ffe... เปลี่ยนทุกครั้ง	0x7fff..., 0x7ffe..., 0x7ffd... ยังเปลี่ยนทุกครั้ง	ไม่ต่าง ตรงไม่ได้
x global ใน display (ไฟล์เดียวกัน)	เปลี่ยนทุกครั้ง	0x404020 ทั้งสามครั้ง	ต่างกัน ตรงที่อยู่ได้
ค่าที่พิมพ์	20 และค่าขยะ	20 และค่าขยะ	ไม่ต่าง

(1) ตัวแปร global ถูกตรงที่อยู่ได้ ด้วยเหตุผลเดียวกับตัวแปร static ในข้อ 1 คือมันอยู่ในส่วน .data ซึ่งเป็นส่วนหนึ่งของไฟล์ เมื่อ linker ผูกที่อยู่จริงไว้ตั้งแต่ตอน link (ไฟล์ชนิด EXEC) เคอร์เนลก็ต้องโหลดลงที่เดิมเสมอ ที่น่า

สนใจคือข้อนี้ให้ผลเหมือนข้อ 1 ทั้งที่ static กับ global เป็นคนละ storage class กัน ซึ่งยืนยันว่าสิ่งที่กำหนดว่า ที่อยู่จะคงที่หรือไม่ ไม่ใช่ *คีย์เวิร์ด* แต่คือ ตัวแปรไปอยู่ส่วนไหน ของหน่วยความจำ

(2) ตัวแปรอัตโนมัติใน main ยังสุมเหมือนเดิม เพราะอยู่บน stack ซึ่งเคอร์เนลเป็นคนสร้างและสุมตำแหน่งให้ ตอนสร้าง process ไม่ได้อยู่ในไฟล์ -no-pie จึงไม่มีอำนาจอะไรกับมัน

(3) ข้อควรระวังในการอ่านตัวเลข ที่อยู่ของ global ในไฟล์ q2_extern_nopie คือ 0x404018 แต่ในไฟล์ q2_noextern_nopie คือ 0x404020 ต่างกัน 8 ไบต์ ตัวเลขที่ไม่เท่ากันนี้ ไม่ได้ ขัดกับข้อสรุป เพราะเป็นคนละไฟล์ ไบนารีกัน (โค้ดต่างกัน linker จึงจัดวางต่างกันเล็กน้อย) สิ่งที่ต้องสรุปพูดถึงคือ ที่อยู่ต้องคงที่เมื่อ *รันไฟล์เดิมซ้ำ* ซึ่งเป็นจริงทั้งสองไฟล์

(4) ค่าที่พิมพ์ไม่เปลี่ยน -no-pie เป็นเรื่องการจัดวาง หน่วยความจำล้วน ๆ ไม่ได้แก้บั๊กการบดบังตัวแปรให้ และไม่ได้ทำให้ตัวแปร ที่ไม่ได้กำหนดค่าเริ่มต้นกลายเป็นศูนย์

2.5 หลักฐานยืนยันคำอธิบาย

หลักฐานที่ 1: x เป็น symbol แบบ global จริง

```
$ nm q2_extern_nopie | grep -w x          # 'D' upper case = external linkage
0000000000404018 D x                      <- exactly what the program printed

$ nm q2_noextern_nopie | grep -w x       # the global x is still there, untouched
0000000000404020 D x                      <- the local x in main() has no symbol

$ readelf -s q2_extern_nopie | grep -w x
21: 0000000000404018      4 OBJECT GLOBAL DEFAULT 25 x
```

Code 19: ตาราง symbol ของไฟล์ -no-pie ทั้งสอง

ตัวอักษร D *ตัวใหญ่* คือจุดที่ต้องเทียบกับข้อ 1 ซึ่งได้ d *ตัวเล็ก* สำหรับตัวแปร static ตัวใหญ่แปลว่า global คือไฟล์อื่นเห็นและใช้ร่วมได้ ส่วนตัวเล็กแปลว่า local คือมีอยู่ในไฟล์นี้เท่านั้น นี่คือ *linkage* ที่จับต้องได้จริงในไฟล์ไบนารี ส่วน readelf ยืนยันเพิ่มว่าเป็น OBJECT ขนาด 4 ไบต์ binding เป็น GLOBAL และอยู่ใน section ที่ 25 ซึ่งก็คือ .data และที่สำคัญคือ ไม่มี symbol ไตเลยสำหรับตัวแปร x ตัวใหม่ใน main เพราะตัวแปรอัตโนมัติไม่ได้อยู่ในไฟล์

หลักฐานที่ 2: คำว่า extern ในฟังก์ชันไม่สร้างโค้ดเครื่องเลยแม้แต่บิตเดียว

ถ้ากล่าวกันว่า “extern ในฟังก์ชันเป็นการเขียนซ้ำซ้อน” เป็นจริง การลบ *ทั้งบรรทัด* นั้นทิ้งไปเลยต้องได้โค้ดเครื่องที่เหมือนกันทุกบิต สคริปต์จึงสร้างไฟล์ q2_nodect1.c ที่ไม่มีบรรทัดประกาศใน main เลย แล้วเทียบ disassembly ของฟังก์ชัน main ระหว่างสองไฟล์

```
$ diff q2_extern.c q2_nodect1.c
21d20
<     extern int x;

$ diff dis_extern.txt dis_nodect1.txt
identical, the extern declaration inside main() generates no code at all

$ cat dis_extern.txt          # the machine code of main() in both files
40117b: endbr64
40117f: push    %rbp
401180: mov     %rsp,%rbp
401183: mov     0x2e8f(%rip),%eax    # 404018 <x>      <- reads the global directly
401189: mov     %eax,%esi
40118b: lea     0xec0(%rip),%rax     # 402052 <_IO_stdin_used+0x52>
401192: mov     %rax,%rdi
401195: mov     $0x0,%eax
40119a: call    401040 <printf@plt>
40119f: lea     0x2e72(%rip),%rax    # 404018 <x>      <- takes its address
...
4011c2: call    401136 <display>
4011c9: ret
```

Code 20: เทียบโค้ดเครื่องของ main() ระหว่างไฟล์ที่มีและไม่มีบรรทัด extern int x;

นอกจากจะเหมือนกันทุกบรรทัดแล้ว โค้ดยังบอกด้วยว่า main อ่านค่าจากที่อยู่ 404018 ซึ่งเป็นตัวแปร global โดยตรง ไม่มีการจองเนื้อที่บน stack ให้ x เลยแม้แต่ไบต์เดียว (ไม่มีคำสั่ง `sub $N,%rsp` สำหรับตัวแปร)

หลักฐานที่ 3: การประกาศไม่ใช่นิยาม ถ้าไม่มีนิยาม linker จะฟ้อง

```
$ diff q2_extern.c q2_nodef.c
10d9
< int x = 20;

$ gcc -w -o q2_nodef q2_nodef.c
ld: q2_nodef.c:(.text+0xa): undefined reference to `x'
ld: q2_nodef.c:(.text+0x27): undefined reference to `x'
ld: q2_nodef.c:(.text+0x4f): undefined reference to `x'
ld: q2_nodef.c:(.text+0x6c): undefined reference to `x'
collect2: error: ld returned 1 exit status
(exit status of gcc = 1 -> the compiler was happy, the linker was not)
```

Code 21: ลบบรรทัด `int x = 20;` ที่เหลือแต่ `extern int x;` ในสองฟังก์ชัน

จุดสำคัญคือ คอมไพเลอร์ผ่านฉลุย ข้อผิดพลาดมาจาก ld ซึ่งเป็น linker ทั้งหมด เพราะ `extern int x;` เป็นการสัญญากับคอมไพเลอร์ว่า “เดี๋ยวมีคนนิยามให้” คอมไพเลอร์จึงปล่อยผ่านและทิ้งช่องว่างไว้ให้ linker เติม พอถึงเวลาเชื่อมไฟล์แล้วหาไม่เจอจึงฟ้องครบทั้ง 4 จุดที่มีการอ้างถึง x (สองจุดใน `display` และสองจุดใน `main`)

หลักฐานที่ 4: ปิด ASLR แล้วที่อยู่หยุดเปลี่ยน (กรณีควบคุม)

```
$ setarch -R ./q2_extern | grep Address # run 1, 2, 3
Address of x (in main function): 0x55555558010
Address of x (in main function): 0x55555558010
Address of x (in main function): 0x55555558010

$ setarch -R ./q2_noextern | grep Address # run 1, 2, 3 (the local x in main)
Address of x (in main function): 0x7fffffffe294
Address of x (in main function): 0x7fffffffe294
Address of x (in main function): 0x7fffffffe294

$ setarch -R bash -c 'grep "[stack\]" /proc/self/maps'
7ffffffde000-7fffffffe000 rw-p 00000000 00:00 0 [stack]
```

Code 22: รันไฟล์ PIE ชุดเดิมด้วย `setarch -R`

ผลเหมือนข้อ 1 ทุกประการ คือทั้งสองโปรแกรมหยุดสุมเมื่อปิด ASLR และที่อยู่ 0x7fffffffe294 ของตัวแปรใน main ตกอยู่ในช่วง [stack] พอดี ยืนยันว่ามันเป็นตัวแปรบน stack จริง

หลักฐานที่ 5: สิ่งที่ extern มีไว้ใช้จริง ๆ

โจทย์ใช้ `extern` ในไฟล์เดียว ซึ่งไม่ใช่สถานการณ์ที่มันถูกออกแบบมาเพื่อแก้ การใช้งานจริงคือให้ไฟล์ .c หลายไฟล์ใช้ตัวแปรตัวเดียวกัน จึงเขียนตัวอย่างสั้น ๆ สองไฟล์เพิ่มไว้เพื่อให้เห็นภาพ

```
1 /* ----- q2_shared_def.c : the DEFINITION Lives here ----- */
2 int shared_counter = 100;
3 void bump_from_other_file(void) { shared_counter++; ... }
4
5 /* ----- q2_shared_main.c : only a DECLARATION ----- */
6 extern int shared_counter;
7 void bump_from_other_file(void);
8 int main(void) { ... bump_from_other_file(); ... }
```

Code 23: `q2_shared_def.c` นิยามตัวแปร ส่วน `q2_shared_main.c` แค่ประกาศ

```
$ gcc -Wall -Wextra -o q2_shared q2_shared_main.c q2_shared_def.c
$ ./q2_shared
q2_shared_main.c: value = 100, address = 0x5f9fe0256010
q2_shared_def.c : value = 101, address = 0x5f9fe0256010
q2_shared_main.c: value = 101, address = 0x5f9fe0256010 (changed by the other file)

$ nm q2_shared_main.o | grep -w shared_counter # 'U' = Undefined, only a declaration
U shared_counter
$ nm q2_shared_def.o | grep -w shared_counter # 'D' = Defined, this is where it lives
0000000000000000 D shared_counter
```

Code 24: ตัวแปรเดียวกันจริง ที่อยู่ตรงกันข้ามไฟล์ และแก้จากอีกไฟล์ได้

ตาราง symbol ของไฟล์ .o ทั้งสองอธิบายกลไกได้ครบในสองบรรทัด ผังที่มีแต่ extern ได้สถานะ u (Undefined) คือ “ขอใช้ชื่อนี้ แต่ไม่รู้ว่ายู่ไหน” ส่วนผังที่นิยามได้สถานะ d พร้อมทั้งอยู่ หน้าที่ของ linker คือจับคู่สองอันนี้เข้าด้วยกัน ผลคือทั้งสองไฟล์พิมพ์ที่อยู่ เดียวกันเป๊ะ และการแก้ค่าจากไฟล์หนึ่งเห็นผลในอีกไฟล์ทันที

2.6 อภิปรายผล

- **extern เป็นคำสั่งที่พูดกับ linker ไม่ใช่กับ CPU:** ต่างจาก static ในข้อ 1 ที่มีผลจริงต่อการจัดวางหน่วยความจำ คำว่า extern ในโค้ดของโจทยไม่ได้ทำให้เกิดคำสั่งเครื่องเพิ่มขึ้น แม้แต่บิตเดียว (หลักฐานที่ 2) มันเป็นเพียงข้อมูลที่คอมไพเลอร์ใช้ตัดสินใจว่า ชื่อนี้หมายถึงอะไร แล้วฝากงานที่เหลือให้ linker ไปหา definition มาให้
- **บทเรียนที่แท้จริงของข้อนี้คือการบดบังตัวแปร (shadowing):** โจทย์บอกให้ “ลบคำว่า extern” ซึ่งฟังดูเหมือนแค่เอาคำที่ไม่จำเป็นออก แต่ผลที่ได้กลับเป็นการสร้างตัวแปรใหม่ที่บังตัวเดิมทั้งดุ้น นี่เป็นบั๊กที่เจอบ่อยมากในโปรแกรมจริง เพราะคอมไพเลอร์ผ่าน ไม่มี error และโปรแกรมยังรันได้ตามปกติ วิธีป้องกันคือเปิด -wshadow เพิ่มจาก -Wall -Wextra ซึ่งจะเตือนทันทีที่ชื่อในบล็อกไปบังชื่อข้างนอก ในกรณีนี้ -Wall ช่วยจับได้อยู่แล้วโดยบังเอิญ ผ่านทาง -Wuninitialized แต่ถ้าโค้ดเขียนว่า `int x = 5;` แทน คำเตือนนั้นจะหายไปและบั๊กจะเงียบสนิท
- **ค่าขยะที่อธิบายได้ ไม่ได้แปลว่าฟังหาได้:** ตารางเทียบค่ากับที่อยู่ แสดงว่าเลข 32765–32767 มีที่มาชัดเจน คือเศษของพอยน์เตอร์ที่ค้างอยู่ใน สล็อตนั้น แต่มาตรฐาน C ระบุว่า การอ่านตัวแปรอัตโนมัติที่ยังไม่กำหนดค่าเป็น undefined behavior ซึ่งแปลว่าคอมไพเลอร์มีสิทธิ์ทำอะไรก็ได้ รวมถึงตัดโค้ดทิ้ง หรือให้ผลต่างกันในแต่ละครั้งที่คอมไพล์ การอธิบายได้จึงมีค่าในแง่การเรียนรู้ ว่าหน่วยความจำถูกใช้ซ้ำอย่างไร ไม่ใช่ในแง่การเขียนโปรแกรมจริง
- **ข้อ 1 กับข้อ 2 เป็นสองแกนที่ตั้งฉากกัน:** static ในข้อ 1 ทำให้ตัวแปร *อายุยืนขึ้น* แต่ *ปิด* การมองเห็นจากไฟล์อื่น ส่วน global กับ extern ในข้อ 2 *เปิด* การมองเห็นให้กว้างที่สุด เห็นได้ชัดที่สุดจากตาราง symbol คือ `d y.o` ตัวเล็ก เทียบกับ `d x` ตัวใหญ่ ทั้งสองตัวอยู่ใน `.data` เหมือนกัน มีที่อยู่คงที่ เมื่อคอมไพล์ด้วย `-no-pie` เหมือนกัน ต่างกันแค่ใครเห็นชื่อมันได้บ้าง
- **ในโปรแกรมจริงไม่ควรเขียน extern ไว้ในไฟล์ .c:** วิธีที่ถูกต้องคือประกาศ `extern int x;` ไว้ในไฟล์ `.h` แล้วให้ทุกไฟล์ที่ต้องใช้ `#include` เข้าไป ส่วน definition `int x = 20;` เขียนไว้ในไฟล์ `.c` เพียงไฟล์เดียว เหตุผลคือถ้าเขียนประกาศซ้ำ ๆ ในหลายไฟล์เอง วันที่ชนิดของตัวแปรเปลี่ยน จะต้องไล่แก้ทุกไฟล์ และถ้าแก้ไม่ครบ linker จะไม่พ้องด้วย เพราะมันจับคู่ ด้วย ชื่อ ไม่ได้ตรวจชนิด ซึ่งจะกลายเป็นบั๊กที่หายาก
- **ตัวแปร global ควรใช้เท่าที่จำเป็น:** ข้อนี้แสดงให้เห็นว่า ตัวแปร global ตัวเดียวถูกอ่านและแก้ไขได้จากทุกฟังก์ชันในโปรแกรม (ตัวอย่างสองไฟล์ในหลักฐานที่ 5 คือภาพชัด ๆ ที่ฟังก์ชันในอีกไฟล์ แก้ค่าแล้วเห็นผลทันที) ซึ่งสะดวกแต่ทำให้ไล่หาสาเหตุของบั๊กยาก เพราะไม่มีทางรู้ว่าใครแก้ค่าไปบ้าง และเมื่อมีหลาย thread ก็เกิด race condition ได้เช่นเดียวกับตัวแปร static ในข้อ 1 แนวทางที่ปลอดภัยกว่าคือส่งค่าผ่านพารามิเตอร์ของฟังก์ชัน หรือถ้าจำเป็นต้องมี global จริง ๆ ก็ประกาศเป็น static ที่ระดับไฟล์เพื่อจำกัดขอบเขตให้แคบที่สุดเท่าที่ทำได้